

Final Year Design Project
AI-Powered Smart Road Visibility & Hazard Detection System



Zartasha Arshad	BCS22001
Isba Sohail	BCS22003
Noor-E-Hira	BCS22018
Ifra Kashif	BCS22020

Under the supervision of
Prof. Dr. Saad Ahmad Khan

Bachelor of Science in Computer Science (2022-2026)

**FACULTY OF COMPUTING &
INFORMATION TECHNOLOGY (FCIT),
UNIVERSITY OF THE PUNJAB, LAHORE.**

AI-Powered Smart Road Visibility & Hazard Detection System

A project presented to
University of the Punjab, Lahore

In partial fulfilment
of the requirement for the degree of

Bachelors of Science in Computer Science (2022-2026)

By

Zartasha Arshad	BCS22001
Isba Sohail	BCS22003
Noor-E-Hira	BCS22018
Ifra Kashif	BCS22020

**DEPARTMENT OF INFORMATION TECHNOLOGY,
UNIVERSITY OF THE PUNJAB,
GUJRANWALA CAMPUS, GUJRANWALA.**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----

Zartasha Arshad [BCS22001]

Signature: -----

Noor-E-Hira [BCS22018]

Signature: -----

Isba Sohail [BCS22003]

Signature: -----

Ifra Kashif [BCS22020]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSCS “AI Powered Smart Road Visibility & Hazard Detection System” was developed by

Zartasha Arshad (BCS22001), Isba Shohail (BCS22003), Noor-E-Hira (BCS22018), and Ifra Kashif (BCS22020) under the supervision of “Dr. Saad Ahmad Khan ” in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Signature: -----

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC))

Signatures		
Name	Dr. Ghulam Mustafa	Prof. Muhammad Rizwan Abbas
	FAC1	FAC2

Signature: -----

Head of FYDP Coordination Office:

Signature:-----

Dated: _____

Chairperson, Department of Information Technology

Executive Summary

The primary objective of ZenDrive is to mitigate road accidents caused by poor visibility conditions such as heavy smog, dense fog, and low-light environments. The project addresses the problem of driver sensory overload and reduced reaction time by providing an automated, AI-driven visibility enhancement and real-time hazard detection system.

The system utilizes a multi-stage approach to ensure driver safety. For visibility enhancement, it employs the CLAHE (Contrast Limited Adaptive Histogram Equalization) dehazing algorithm to process live camera feeds. Hazard detection is performed using a fused YOLOv8 Nano model, specifically optimized for speed and real-time performance on desktop environments. To prevent alert fatigue, the system implements a thread-safe audio alert mechanism with a 4-second cooldown, providing spoken warnings about detected obstacles. The backend infrastructure is built on CockroachDB Cloud, ensuring a scalable and reliable data logging system for hazard events and trip sessions, accessible through a centralized Admin Dashboard.

Testing indicates that ZenDrive significantly improves the clarity of visual inputs in smoggy conditions and provides highly accurate hazard detection with minimal latency. By integrating automated dehazing with intelligent audio alerts, the system offers a robust safety layer for drivers. The implications of this project suggest a viable, software-based alternative to expensive thermal imaging hardware for standard vehicles.

Keywords: Computer Vision, YOLOv8, Dehazing Algorithm, Real-time Hazard Detection, Smart Driver Assistance.

Acknowledgement

The completion of this Final Year Project was made possible through the support and guidance of several individuals. We wish to express our deepest gratitude to our supervisor, Dr. Saad Ahmed Khan, for his invaluable technical insights into automata theory, computer vision, and his constant encouragement throughout the development of ZenDrive.

Special thanks to our friends and families for their patience and support during the long hours of coding and testing. Finally, we acknowledge the developers of the open-source libraries, specifically the Ultralytics and OpenCV communities, whose tools were instrumental in building this system.

Signature: -----

Zartasha Arshad [BCS22001]

Signature: -----

Noor-E-Hira [BCS22018]

Signature: -----

Isba Sohail [BCS22003]

Signature: -----

Ifra Kashif [BCS22020]

Abbreviations

Table of all the abbreviations and acronyms used in our FYP along with their respective brief description. The abbreviation list is ordered alphabetically.

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CLAHE	Contrast Limited Adaptive Histogram Equalization
CPU	Central Processing Unit
CSV	Comma Separated Values
DB	Database
DIP	Digital Image Processing
FPS	Frames Per Second
FR	Functional Requirement
GUI	Graphical User Interface
HUD	Heads-Up Display
ML	Machine Learning
NFR	Non-Functional Requirement
OS	Operating System
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
UX	User Experience
YOLO	You Only Look Once (Object Detection Algorithm)

Table of Contents

Contents

1. Introduction	14
1.1 Problem Statement	14
1.2 Problem Solution	14
1.3 Objectives of the Proposed System	14
1.4 Scope	14
1.5 System Components	14
1.5.1 Module 1: AI Perception & Enhancement Engine	15
1.5.2 Module 2: Driver Dashboard (Desktop Client)	15
1.5.3 Module 3: Admin Web/Desktop Panel	15
1.6 Related System Analysis/Literature Review	15
1.7 Vision Statement:	16
1.8 System Limitations and Constraints:	16
1.9 Tools and Technologies	16
1.10 Project Deliverables.....	17
1.11 Project Planning	17
1.12 Summary	18
2. Analysis	20
2.1 User Classes and Characteristics	20
2.2 Requirement Identifying Technique.....	20
2.3 Functional Requirements.....	21
2.3.1 Functional Requirement X	24
2.4 Non-Functional Requirements	28
2.4.1 Reliability	28
2.4.2 Usability	28
2.4.3 Performance	28
2.4.4 Security	28
2.5 External Interface Requirements.....	29
2.5.1 User Interfaces Requirements	29
2.5.2 Software Interfaces	29
2.5.3 Hardware Interfaces	29
2.5.4 Communications Interfaces	29
2.6 Summary	30
3. System Design	32

3.1 Design Considerations.....	32
3.2 Design Models:.....	32
3.3 Architectural Design:	32
3.3.1 UML Component Diagram:	33
3.3.2 UML class Relationship Diagram:	34
3.4 Data Design:	36
3.4.2 Data Dictionary:	39
3.5 User Interface Design:.....	42
3.5.1 Screen Images:	42
3.5.2 Screen Objects and Actions:	43
3.6 Behavioral Model:	44
3.6.1 Interaction Diagram:.....	44
3.6.2 State Transition Diagram:	45
3.7 Design Decisions:	45
3.7.1 Selection of AI Architecture: YOLOv8 Nano.....	45
3.7.2 Database Selection: CockroachDB with SQLite Fallback.....	46
3.7.3 Visibility Algorithm: Triple-Pass DIP Pipeline	46
3.7.4 Selection of Programming Language: Python	46
3.7.5 Deployment Architecture: Standalone Desktop Application	47
3.8 Summary:.....	47
4. Implementation	49
4.1 Algorithm	49
Output: Alerts.....	50
4.2 External APIs/SDKs.....	50
4.3 Code Repository	51
4.3.1 Metrics of the Git Repository	51
4.4 Summary	51
5. Introduction	53
5.1 Unit Testing (UT).....	53
5.2 Functional Testing (FT)	53
5.3 Integration Testing (IT)	54
5.4 Performance Testing (PT)	55
5.5 Summary	55
6. Introduction:	57
6.1 Conversion Method	57
6.2 Deployment:	57
6.2.1 Data Conversion:.....	58
6.2.2 Training :	58

6.3 Post Deployment Testing:	59
6.4 Challenges :	60
6.5 Summary:	61
7. Introduction	63
7.1 Evaluation	63
7.2 Traceability Matrix	63
7.3 Conclusion	63
7.4 Future Work	64
References	64
Appendix-A Use Case Description (Fully Dressed Format)	66
Appendix-B General Coding Standards & Guidelines	69
Appendix-C Application Prototype	71

List of Figures

Figure 1: work gantt chart.....	17
Figure 2: work distrubution	18
Figure 3: UML COMPONENT DIAGARM	33
Figure 4: UML Class-Relationship Diagram.....	34
Figure 5: ER DIAGRAM.....	38
Figure 6: splash page.....	42
Figure 7: registration page	42
Figure 8: login page	43
Figure 9: detection output	43
Figure 10: setting page.....	43
Figure 11: STATE DIAGRAM	45

List of Tables

Table 1: Related System Analysis with proposed project solution.....	15
Table 2: Tools and Technologies for Proposed Project.....	16
Table 3: user class and characteristic.....	20
Table 4: UseCase 1(Monitoring Real Time Visibility,UC1).....	21
Table 5: UseCase 2(Alert Drivers to Critical Hazards,UC2).....	22
Table 6: UseCase 3(Configure Safety Settings,UC3).....	23
Table 7: UseCase 4(Manage Trip Data Logging,UC4).....	23
Table 8: Feature 1(User Authentication).....	24
Table 9: Feature 2(Real-time Dehazing).....	25
Table 10: : Feature 3(Visibility Meter).....	25
Table 11: Feature 4(Hazard Detection).....	26
Table 12: Feature 5(UI Customization).....	26
Table 13: Feature 6(Trip Data Logging).....	27
Table 14: Feature 7(Audio Customization).....	27
Table 15 : UML Class-Relationship Table.....	35
Table 16: ENTITY 1: USERS.....	36
Table 17: ENTITY 2: TRIPSESSIONS.....	37
Table 18 : Entity 3: HazardEvents.....	37
Table 19: DATA DICTIONARY TABLE.....	42
Table 20: Screen Objects and Actions.....	44
Table 21: Algorithm 1: Real-Time Visibility and Hazard Detection.....	49
Table 22: Algorithm 2: Visibility Estimation.....	50
Table 23: Algorithm 3: Alert Generation.....	50
Table 24: Details of APIs used in the project.....	50
Table 25: Unit Test Case (UT1).....	53
Table 26: Unit Test Case (UT2).....	53
Table 27: Functional Test Case (FT1).....	54
Table 28: Functional Test Case (FT2).....	54
Table 29: Integration Test Case (IT1).....	54
Table 30: Performance Test Case (PT1).....	55
Table 31: Performance Test Case (PT2).....	55
Table 32 : Evaluation Status.....	63
Table 33: traceability matrix show.....	63
Table 34 : Objectives vs System Functionality.....	64

Chapter 1

Introduction

1. Introduction

This chapter provides the overview of the project. The first paragraph provides the chapter introduction – presenting the purpose of the chapter and how this chapter contributes to achieve project goals.

1.1 Problem Statement

ZenDrive addresses the high incidence of road accidents caused by severe winter smog in Pakistani urban environments. Currently, environmental factors specifically smog and dense fog, significantly reduce road visibility, leading to an increase in vehicular accidents and fatalities. High-end safety sensors like LIDAR are too expensive for mass-market adoption. This system is developed to provide a software-based "second sight" that identifies hazards through extreme visual obstructions.

1.2 Problem Solution

The software implements a real-time computer vision pipeline that dehazes live video feeds and identifies obstacles such as cars, pedestrians, and cyclists.

- Provide a reliable Visibility Meter (Good/Moderate/Critical).
- Maintain a processing speed of 15–30 FPS on standard hardware.
- Apply digital image enhancement to restore visual clarity.

1.3 Objectives of the Proposed System

- **Be Specific:** Implement a YOLOv8-based detection model specifically tuned for low-confidence hazy inputs.
- **Be Measurable:** Maintain a processing speed of at least 15 FPS on standard consumer-grade laptops.
- **Be Achievable:** Utilize the YOLOv8-nano model for efficient, real-time inference on non-GPU optimized hardware.
- **Be Relevant:** Provide a low-cost safety alternative for drivers in high-smog regions like Punjab.
- **Be Time-bound:** Complete full cloud-synchronization and the industrial-level prototype by the graduation cycle of 2026.

1.4 Scope

Scope includes real-time video dehazing, object detection, visibility scoring, and automated auditory alerts. It specifically covers the development of a Driver Dashboard and an Admin Panel while excluding direct autonomous vehicle control.

1.5 System Components

The ZenDrive system is organized into distinct functional modules designed to ensure real-time perception, data integrity, and cross-platform accessibility. These modules are grouped according to their operational roles within the desktop and cloud environment:

1.5.1 Module 1: AI Perception & Enhancement Engine

This module serves as the computational core of the project, handling all real-time video processing.

- **Triple-Pass Dehazing:** Implementation of CLAHE, Unsharp Masking, and Gamma Correction to clear atmospheric noise.
- **Hazard Detection:** YOLOv8-nano inference to identify vehicles, pedestrians, and cyclists through up to 4 layers of obstruction.
- **Visibility Scoring:** Contrast-based analysis to provide a real-time "Good," "Moderate," or "Critical" visibility status.
- **Auditory Feedback:** Integration with native OS speech engines to provide non-blocking voice alerts.

1.5.2 Module 2: Driver Dashboard (Desktop Client)

The primary interface for the end-user, optimized for safety and minimal distraction.

- **Live Enhanced Feed:** Displays the dehazed video stream with augmented reality bounding box overlays.
- **Real-Time HUD:** A high-contrast Heads-Up Display showing the Visibility Meter and system FPS.
- **Session Control:** Features to start and stop trip sessions, which trigger cloud data entry.
- **Accessibility Settings:** Options to customize font sizes, UI themes (Dark Mode), and audio alert tones.

1.5.3 Module 3: Admin Web/Desktop Panel

A management interface designed for fleet monitoring and historical data review.

- **Cloud Synchronization:** Real-time viewing of trip logs pushed to the CockroachDB cluster.
- **Hazard Analytics:** Detailed list of historical incidents including hazard type and confidence scores.
- **Data Export:** Ability to export trip history into .CSV format for traffic authority reporting.

1.6 Related System Analysis/Literature Review

TABLE 1: RELATED SYSTEM ANALYSIS WITH PROPOSED PROJECT SOLUTION

Application Name	Weakness	Proposed Project Solution
LIDAR Systems	Prohibitive costs for mass adoption.	Low-cost computer vision solution using standard cameras.

Basic Dehazing Apps	Lack real-time hazard identification and cloud sync.	Integrated YOLOv8 detection with CockroachDB cloud synchronization.
---------------------	--	--

1.7 Vision Statement:

For safety-conscious drivers in smog-prone regions **who** face life-threatening low-visibility conditions, **the ZenDrive system is** an AI-powered visibility assistant **that** restores visual clarity and detects hidden hazards. **Unlike** expensive hardware-integrated safety packages, **our product** provides an industrial-grade, cloud-synchronized solution.

1.8 System Limitations and Constraints:

Limitations (External Factors):

- **Extreme Low Light:** Without an external light source (like headlights), detection accuracy depends entirely on the camera's ISO sensitivity.
- **Network Dependency:** Real-time synchronization with CockroachDB is limited by the availability of cellular or Wi-Fi internet.
- **High-Opacity Smog:** Environmental conditions exceeding the "4-layer" simulated threshold may reduce the model's confidence levels.

Constraints (Self-Imposed/Internal Factors):

- **Processing Latency:** The system must maintain an end-to-end processing loop of less than 200 milliseconds to ensure driver safety.
- **Hardware Compatibility:** The software is strictly optimized for consumer-grade laptops (Core i5, 8GB RAM) to maintain its low-cost advantage.
- **Resolution Limit:** Video input is constrained to 640x480 (VGA) to maximize the frame rate (FPS) on CPU-based systems.

1.9 Tools and Technologies

TABLE 2: TOOLS AND TECHNOLOGIES FOR PROPOSED PROJECT

Tools	Version	Rationale
Python	3.10	Primary language for AI perception and UI development.
YOLOv8-nano	v8	Optimized for real-time CPU performance.
CockroachDB	Cloud	Distributed SQL for online, synchronized data.

Iriun Webcam	SDK	Mobile-to-PC HD video streaming interface.
--------------	-----	--

1.10 Project Deliverables

The following items constitute the final deliverables for the ZenDrive project:

1. **Project Planning Document:** Detailed roadmap and resource allocation.
2. **System Requirements Specification (SRS):** Formalized functional and non-functional requirements.
3. **Software Design Specification (SDS):** Architectural blueprints and data design.
4. **Functional Prototype:** The Python-based application integrated with YOLOv8 and CockroachDB.
5. **Final Project Report:** A comprehensive document detailing development, testing, and evaluation.

1.11 Project Planning

The project timeline was managed using a Gantt Chart to track milestones across the four-member team (Zartasha, Isba, Noor-e-Hira, and Ifra). Key milestones included:

- **AI Training & DIP Refinement:** Finalizing the Triple-Pass logic for 4-layer obstruction.
- **UI/UX Development:** Creating the high-contrast dashboard in CustomTkinter.
- **Database Migration:** Transitioning from SQLite to CockroachDB for global synchronization.
- **Validation & Integration Testing:** Ensuring <200ms latency across the full system loop.

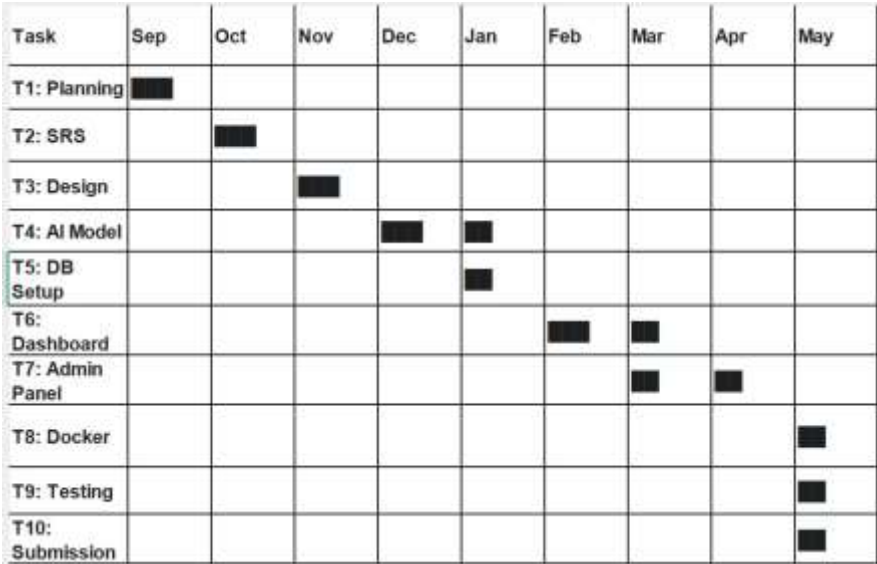


FIGURE 1: WORK GANTT CHART

Task ID	Task Description	Start Date	End Date	Duration	Key Milestone
T1	Project Initiation & Vision	01-Sep-25	20-Sep-25	3 Weeks	Vision Approved
T2	Requirement Analysis (SRS)	21-Sep-25	20-Oct-25	4 Weeks	SRS Finalized
T3	System Design (SDS/UI)	21-Oct-25	20-Nov-25	4 Weeks	Design Models Ready
T4	AI Model Training (YOLOv8)	21-Nov-25	05-Jan-26	6 Weeks	Optimized Weights
T5	Cloud DB Setup (CockroachDB)	06-Jan-26	31-Jan-26	4 Weeks	DB Connectivity
T6	Core Driver Dashboard Dev	01-Feb-26	15-Mar-26	6 Weeks	Alpha Build
T7	Admin Panel & Sidebar Feed	16-Mar-26	05-Apr-26	3 Weeks	Full Monitoring
T8	Docker & Containerization	03-May-26	05-May-26	3 Days	Portable Image
T9	System Testing & QA	10-Apr-26	04-May-26	4 Weeks	Bug-Free Beta
T10	Final Report & Submission	02-May-2026	05-May-26	1 Week	Final Submission

FIGURE 2: WORK DISTRUBUTION

1.12 Summary

Chapter 1 established the foundation for ZenDrive as a low-cost, AI-powered solution for smog-related road safety. It introduced the project's specific objectives—such as real-time 15-30 FPS performance and cloud-synced hazard logging—and defined the modular components that make the system a viable industrial tool. By addressing the critical problem of visual degradation, this chapter highlights the system's importance in providing drivers with life-saving perception capabilities in hazardous atmospheric conditions.

Chapter 2

Requirements Analysis

2. Analysis

This chapter presents the detailed requirements analysis of the ZenDrive system. It identifies the different user classes, their characteristics, and the techniques used to gather system requirements. The chapter further defines the functional and non-functional requirements that guide the system's development, ensuring that it meets performance, usability, and safety standards. Additionally, it outlines external interface requirements, including user interfaces, software, hardware, and communication interactions. This analysis serves as the foundation for the system design and implementation phases.

2.1 User Classes and Characteristics

The following table identifies the primary user classes for ZenDrive and describes the pertinent characteristics that influence system design.

TABLE 3: USER CLASS AND CHARACTERISTIC

User Class	User Characteristics
Private Commuter	Frequency: Daily (Morning/Evening commute). Tech Skill: Low to Medium. Key Needs: Simple UI and clear audio/visual alerts to ensure personal safety without distraction.
Commercial Driver	Frequency: High (Long hours on highways). Tech Skill: Low. Key Needs: High reliability and minimal interaction to protect fleet vehicles and reduce insurance risks.
Traffic Administrator	Frequency: Occasional (Data Review). Tech Skill: Medium to High. Key Needs: Access to aggregated trip logs and visibility data to monitor smog severity and public safety trends.

2.2 Requirement Identifying Technique

To derive the functional requirements for **ZenDrive**, we utilized a combination of analytical and interactive techniques.

- **Document Analysis:** We analyzed existing literature on road safety in Pakistan to identify gaps in current solutions, establishing the need for a low-cost solution specifically for smog.
- **Use Case Analysis:** Since ZenDrive is an interactive application, this was our primary technique. We mapped out interactions for actors like the Private Commuter and the System Administrator to define system behaviors for "Critical" visibility levels.
- **Storyboarding:** To ensure the interface is non-distracting for drivers, we sketched the sequence of events from the Splash Screen to the Main Dashboard, visualizing the flow of warnings.

2.3 Functional Requirements

Use Case 1: Monitor Real-Time Visibility:

TABLE 4: USECASE 1(MONITORING REAL TIME VISIBILITY,UC1)

UC Identifier	UC1
Requirements Traceability	FR-2, FR-3, FR-4
Purpose	To continuously process the camera feed, quantify the smog density, and display the current Visibility Score on the dashboard to the driver.
Priority	High
Preconditions	The user is authenticated/logged in. The system application is open and initialized. The camera feed is active and providing frames.
Post conditions	The Main Dashboard is actively displayed, showing the real-time video feed and the updated Visibility Meter (FR-3).
Actors	Primary Driver, Commercial Driver, External Camera
Extends	N/A
Main Success Scenario	The Driver views the Main Dashboard, initiating the monitoring loop. System captures a new frame from the External Camera (FR-2). System applies image enhancement algorithms (dehazing/contrast) to the frame. System calculates the Visibility Score based on image features and updates the score. System displays the score on the Visibility Meter (e.g., "Good," "Moderate," or "Poor"). System repeats the process starting from Step 2, maintaining 15-30 FPS (FR-4).
Alternate Flow	Low FPS: If processing speed drops below 15 FPS (NFR-Performance), the system displays a warning message to the user/administrator.
Exception	Camera Disconnect: If the video stream stops, the system displays a distinct "Signal Lost" error and automatically attempts to reconnect every 5 seconds (NFR-Fail-Safe).
Includes	Real-Time Video Processing

Use Case 2: Alert Driver to Critical Hazard:
TABLE 5: USECASE 2(ALERT DRIVERS TO CRITICAL HAZARDS,UC2)

UC Identifier	UC2
Requirements Traceability	FR-3, FR-4, FR-7
Purpose	To provide an immediate, high-priority audio and visual warning when a critical low-contrast hazard is detected or visibility drops below the safety threshold.
Priority	High
Preconditions	UC1 is successfully running and providing Visibility Scores (FR-3). Hazard Detection Module (YOLOv8) is initialized and running (FR-4).
Post conditions	The hazard event is logged; the system returns to the continuous monitoring state (UC1).
Actors	Primary Driver, Commercial Driver
Extends	Monitor Real-Time Visibility (UC1)
Main Success Scenario	UC1 is running, and the Visibility Score is "Poor." Hazard Detection Module (YOLOv8) identifies a low-contrast obstacle (e.g., stopped car/pedestrian) within the critical zone. System initiates Audio Alert (voice tone "Immediate hazard detected!") and Visual Alert (flashing red boundary, bounding box) (FR-7). Alert persists until the hazard leaves the frame or the driver manually dismisses the alert. 5. System logs the hazard event details (location, time) to the Trip Data Log (FR-6).
Alternate Flows	Visibility Threshold Alert: If Visibility Score drops to "Critical" (e.g., below 50 meters) but no specific hazard is detected, the system triggers a continuous " Critical Visibility Warning " (less intense visual/audio) until the score improves.
Exception	False Positive Alert: If the alert is triggered by non-obstacle interference (e.g., a glare), the driver can hit the 'Mute' button, logging the event as a suspected false positive.
Includes	N/A

Use Case 3: Configure Safety Settings:

TABLE 6: USECASE 3(CONFIGURE SAFETY SETTINGS,UC3)

UC Identifier	UC3
Requirements Traceability	FR-5, FR-7
Purpose	To allow the user (Driver or Administrator) to adjust safety thresholds, display themes, and audio output preferences.
Priority	Medium
Preconditions	User is authenticated. The system is paused or running in the background.
Post conditions	New settings are saved to the local configuration file and applied immediately upon returning to the Main Dashboard.
Actors	Primary Driver, System Administrator
Extends	N/A
Main Success Scenario	User selects the Settings Menu option. User navigates to the Audio Management tab. User modifies the Voice Pitch (FR-7) via a slider and plays a sample warning message. User navigates to the General tab and toggles the UI to Dark Mode (FR-5). User clicks "Save & Exit." 6. System saves settings and returns to the Main Dashboard, applying the new configuration.
Alternate Flows	Invalid Input: If the user attempts to set an audio pitch value outside the allowed range (e.g., > 1.5x), the system displays an error message ("Value out of range") and reverts to the last saved value.
Exceptions	None
Includes	N/A

Use Case 4: Manage Trip Data Logging:

TABLE 7: USECASE 4(MANAGE TRIP DATA LOGGING,UC4)

Field	Description
Use Case ID	UC4
Use Case Name	Manage Trip Data Logging
Requirements Traceability	FR-3, FR-6

Actors	System Administrator
Description	To allow the Administrator to access, review, and export accumulated visibility and hazard event data from the cloud for fleet analysis.
Trigger	Administrator selects the "Trip History" or "Admin Panel" tab.
Preconditions	User is authenticated as an Administrator; CockroachDB Cloud cluster is active and contains synchronized trip logs.
Post conditions	The trip data is successfully exported as a .CSV file, or the history view is closed.
Normal Flow	Administrator selects the Trip History tab from the main menu. System queries the CockroachDB Cloud database (FR-3) and displays a list of synchronized trip sessions. Administrator selects a specific user or time range and clicks " Export Data ". 4. System generates and saves a .CSV file containing the log data (Timestamp, Visibility Score, Hazard Count) to a local directory.
Alternative Flows	No Cloud Data: If the cloud query returns no logs, the system displays "No Trip Data Available in Selected Range".
Business Rules	Only users with "Admin" privileges can access the global fleet data; standard drivers can only view their own local logs.
Assumptions	The Administrator has an active internet connection to reach the CockroachDB cluster.

2.3.1 Functional Requirement X

Feature 1: User Authentication(Login):

This Feature will start a session with the user . This is a new Feature we added.

TABLE 8: FEATURE 1(USER AUTHENTICATION)

Field	Description
Identifier	FR-1
Title	User Login and Session Management
Requirement	The system shall provide a login interface requiring a valid username and password before granting access to the main dashboard. Upon successful authentication, the system shall initiate a "Trip Session" to record visibility data and hazard events.
Source	Project Team (New Requirement)
Rationale	To ensure secure access to the system and associate trip data (visibility levels and detected hazards) with specific users for monitoring and analysis purposes.
Priority	Medium

Feature 2: Real-Time Dehazing and Visualization:

This Feature will cover the “Dehazed Video” and our “Main Screen”.

TABLE 9: FEATURE 2(REAL-TIME DEHAZING)

Field	Description
Identifier	FR-2
Title	Real-Time Video Dehazing and Display
Requirement	The system shall capture live video from the connected camera and apply an image enhancement algorithm (dehazing/contrast adjustment) to the frames. The processed "clearer" video must be displayed on the Main Dashboard with a latency of less than 100ms.
Source	Proposal Objectives (Visibility Quantification)
Rationale	Drivers cannot see through smog; the system must digitally "clean" the image to reveal road details that the human eye might miss.
Priority	High

Feature 3: Visibility Meter:

This Feature corresponds to our Project’s “Visibility Quantification” Objective.

TABLE 10: : FEATURE 3(VISIBILITY METER)

Field	Description
Identifier	FR-3
Title	Real-Time Visibility Scoring and Display
Requirement	The system shall analyze the video frame to calculate a Visibility Score based on contrast and fog density. This score must be displayed on the Main Dashboard as a graded status (e.g., "Good," "Moderate," "Critical") updated at least once per second.
Source	Proposal Objectives (Visibility Quantification) 2
Rationale	Drivers need an objective measure of safety, as human eyes adjust slowly to worsening conditions.
Priority	High

Feature 4: Hazard Detection:

This Feature corresponds to our Project's "Hazard Detection" Objective.

TABLE 11: FEATURE 4(HAZARD DETECTION)

Field	Description
Identifier	FR-4
Title	Low-Contrast Obstacle Detection
Requirement	The system shall use the optimized YOLOv8 model to detect vehicles, pedestrians, and large obstacles. When a hazard is detected, the system must draw a bounding box around the object on the Main Dashboard and trigger an audio warning if the object is within a critical range.
Source	Proposal Objectives (Hazard Detection Accuracy) 4
Rationale	To prevent collisions with stationary or slow-moving objects that are invisible in smog.
Priority	High

Feature 5: UI Customization:

This Feature will allow the user to Change the UI themes and fonts to increase readability.

TABLE 12: FEATURE 5(UI CUSTOMIZATION)

Field	Description
Identifier	FR-5
Title	UI Customization (Font & Theme)
Requirement	The system shall provide a Settings Menu that allows the user to: Toggle between Light Mode and Dark Mode (Night Mode). 2. Adjust the Font Size of warning text (Small, Medium, Large) for better readability.
Source	User Request (Accessibility Features)
Rationale	To ensure the driver is not blinded by a bright screen at night and can read warnings quickly without squinting.
Priority	Low

Feature 6: Data Logging:

This new Feature will cover the Database/Logging that can be used by Traffic Authorities for smog Severity and Accidents ratio for public safety Policy.

TABLE 13: FEATURE 6(TRIP DATA LOGGING)

Field	Description
Identifier	FR-6
Title	Trip Data Logging and Synchronization
Requirement	The system shall maintain a local SQLite database for immediate trip logging and synchronize this data with the CockroachDB Cloud cluster. Each log must record the Date, Time, Duration, Visibility Scores, and Hazard Event counts. This data shall be accessible to the user and authorities via a "History" tab and the Admin Panel.
Source	Stakeholder Interest (Government/Traffic Police)
Rationale	To provide globally synchronized, verifiable data on smog severity and accident risks to government authorities for safety auditing and public policy development.
Business Rule	Data must be hashed and encrypted before cloud transmission; local logs must persist regardless of internet connectivity status.
Dependencies	FR-1 (User Authentication), FR-3 (Cloud Synchronization)
Priority	Medium

Feature 7: Audio Customization:

TABLE 14: FEATURE 7(AUDIO CUSTOMIZATION)

Field	Description
Identifier	FR-7
Title	Audio Alert & Voice Tone Customization
Requirement	The system shall provide an Audio Settings module that allows the user to: Select Voice Gender: Toggle between Male and Female voice profiles. Adjust Voice Tone/Pitch: Modify the pitch frequency (Low/Deep to High/Sharp) to suit hearing preference. Select Alert Style: Choose between "Assertive" (faster, louder) or "Calm" (slower, softer) speaking styles. 4. Test Audio: Play a sample warning message to verify the selected tone.
Source	User Request (Accessibility & UX Enhancement)
Rationale	To prevent "alarm fatigue" and ensure the audio warning is distinct and audible for drivers with varying hearing capabilities (e.g., presbycusis).
Priority	Low (Enhancement)

2.4 Non-Functional Requirements

This section specifies the non-functional quality standards and performance benchmarks for **ZenDrive**, ensuring the system is reliable, efficient, and secure under operational conditions.

2.4.1 Reliability

- **Availability:** The system shall maintain an uptime of 99.9% during active driving sessions, operating continuously without crashing for typical commute durations (up to 4 hours).
 - **Fail-Safe Mechanism:** In the event of an internet outage, the system must automatically fall back to local **SQLite** storage to prevent data loss.
 - **Error Detection:** If the camera feed is interrupted, the system shall display a "Signal Lost" warning and attempt reconnection every 5 seconds.
 - **Consequences of Failure:** Software failure leads to a loss of driver assistance; therefore, the system is designed to isolate the AI processing from the GUI to prevent total system freezes.
-

2.4.2 Usability

- **Accessibility:** All warning text (e.g., "CRITICAL") must be legible from a standard driver-to-dashboard distance of 60cm.
 - **Ease of Use:** The transition from user login to live tracking must require no more than 3 user interactions (clicks).
 - **User Experience (UX):** The interface shall utilize a high-contrast "Dark Theme" by default to prevent cabin glare and ensure readability in hazy conditions.
 - **Error Recovery:** Users shall be able to dismiss false positive alerts with a single mute button to minimize cognitive load while driving.
-

2.4.3 Performance

- **Processing Latency:** The end-to-end time delay between a real-world event and the system alert shall not exceed **200 milliseconds**.
 - **Frame Rate:** The system must maintain a processing speed of **15–30 Frames Per Second (FPS)** on standard consumer laptops.
 - **Resource Utilization:** The application shall consume no more than 70% of available CPU resources to prevent hardware overheating during extended use.
 - **Initialization Time:** The AI model (YOLOv8-nano) must load into system memory in under 5 seconds.
-

2.4.4 Security

- **Data Privacy:** All user passwords must be hashed using SHA-256 before storage in the **CockroachDB** cloud or local database.

- **Access Control:** Access to historical trip logs and exported CSV files is restricted to authenticated users only.
- **Connection Security:** Communication with the **CockroachDB** cluster is performed via secure HTTPS/SQL protocols to protect database connection strings.

2.5 External Interface Requirements

2.5.1 User Interfaces Requirements

- **GUI Standards:** The system follows a high-contrast dark aesthetic optimized for cockpit environments.
- **Layout Constraints:** The interface is designed for a target resolution of 640x480 to maximize processing efficiency.
- **HUD Elements:** A semi-transparent Heads-Up Display (HUD) must display the Visibility Meter (Green/Yellow/Red) in the top-right zone.
- **Visual Feedback:** Detected hazards shall be highlighted with blue Bounding Boxes (RGB:255,50,50) including class labels and confidence scores.

2.5.2 Software Interfaces

- **Operating Systems:** Compatible with Windows 10/11 (64-bit) and Linux (Ubuntu 20.04 LTS).
- **Libraries:** Interfaces with **OpenCV 4.8+** for image processing and **Ultralytics YOLOv8** for AI inference.
- **Database Management:** Connects to **CockroachDB Cloud** for online synchronization and uses **SQLite3** for local persistent storage.

2.5.3 Hardware Interfaces

- **Video Input:** Supports USB 2.0/3.0 Webcams and mobile device cameras via the **Iriun Webcam SDK**.
- **Protocols:** Utilizes DirectShow (Windows) or V4L2 (Linux) for camera interaction.
- **Industrial Scaling:** Designed to support future hard-wired **OEM integration** for car-integrated infotainment screens.

2.5.4 Communications Interfaces

- **Network Protocols:** Uses HTTPS for secure communication with the cloud-based database cluster.
- **Data Export:** Provides an interface to export Trip Logs into **.CSV** format for analysis by traffic authorities.

- **Internal Sync:** Manages data flow between the local detection module and the remote **CockroachDB** instance using standard SQL queries.

2.6 Summary

Chapter 2 detailed the rigorous requirements engineering process for **ZenDrive**, identifying key user classes such as Private Commuters and Fleet Administrators. By defining seven core Functional Requirements and high-performance Non-Functional benchmarks (e.g., <200ms latency and 15–30 FPS), the system is architected to meet safety-critical standards. The integration of diverse external interfaces—ranging from mobile sensors to cloud databases—ensures that **ZenDrive** provides a reliable, secure, and user-friendly assistant for driving in low-visibility smog and fog conditions.

Chapter 3

System Design

3. System Design

This chapter details the transformation of the system requirements into a modular, structural design, focusing on the architectural blueprint of the **ZenDrive** system.

3.1 Design Considerations

- **Assumptions and Dependencies:**
 - The system assumes that the host hardware (Mac/Windows) includes a functional webcam or dashcam recognized by the OS.
 - It depends on the Ultralytics YOLOv8 library and the specific yolov8n.pt model weights for object detection.
 - The system assumes sufficient thermal overhead for continuous high-speed AI inference.
- **Limitations:**
 - Visibility is calculated via contrast analysis rather than depth sensors, providing clarity but not absolute distance measurements.
 - Detection accuracy in total darkness relies heavily on the camera's ISO sensitivity and external light sources (e.g., headlights).
- **Risks:**
 - **System Lag:** Mitigated by decoupling the AI Processing Layer from the Presentation Layer to keep the GUI responsive.
 - **Data Consistency:** Mitigated by utilizing **CockroachDB** for synchronized cloud storage and **SQLite** for atomic offline transactions.

3.2 Design Models:

3.3 Architectural Design:

The system utilizes a Layered Architecture. This pattern was chosen to ensure modularity and separation of concerns, allowing the AI components to be upgraded independently of the User Interface.

3.3.1 UML Component Diagram:

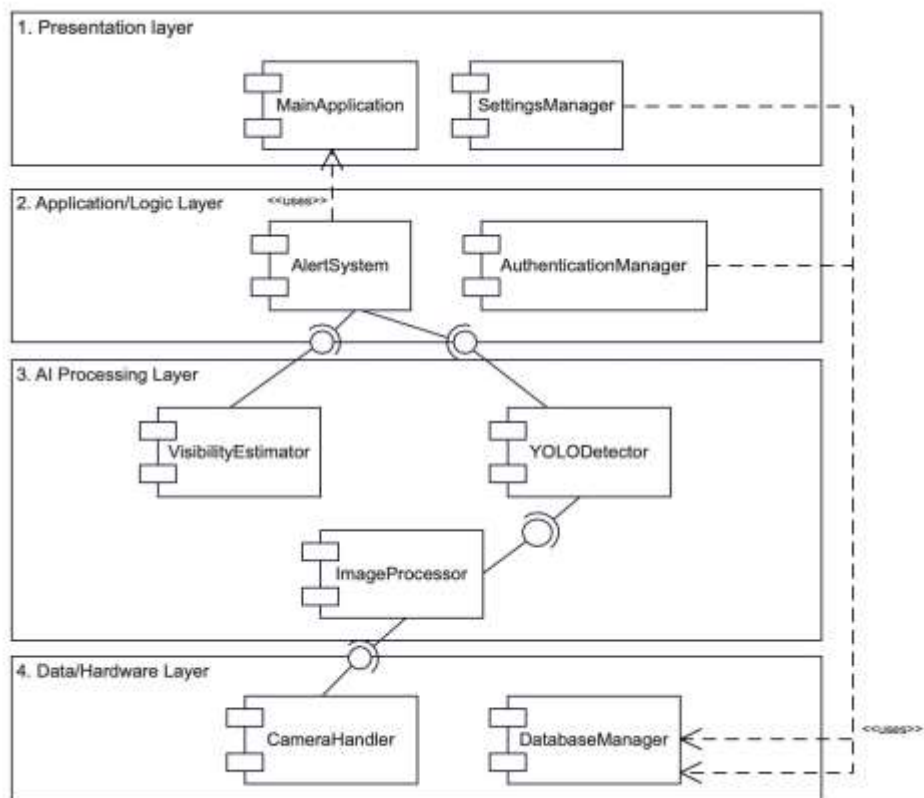


FIGURE 3: UML COMPONENT DIAGARM

Description:

1. Presentation Layer:

- **Components:** MainApplication, SettingsManager.
- **Responsibility:** Manages the GUI lifecycle, captures user inputs, and displays the processed video feed. It communicates only with the Application Layer.

2. Application / Logic Layer:

- **Components:** AlertSystem, AuthenticationManager.
- **Responsibility:** Acts as the controller. It enforces business logic (e.g., verifying passwords, deciding when to trigger an alert based on thresholds).

3. AI Processing Layer:

- **Components:** YOLODetector, VisibilityEstimator, ImageProcessor.
- **Responsibility:** The computational core. It receives raw frames and returns structured data (object lists, visibility scores) to the Logic Layer.

4. Data / Hardware Layer:

- **Components:** CameraHandler, DatabaseManager.
- **Responsibility:** Abstracts the low-level details of hardware interactions (webcam drivers) and persistent storage (SQL queries).

3.3.2 UML class Relationship Diagram:

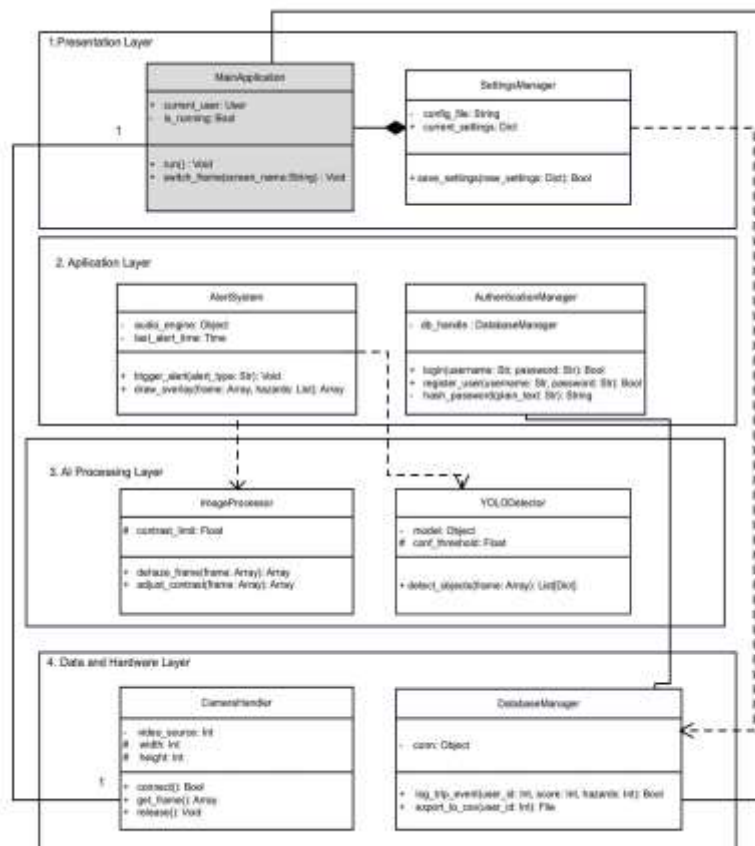


FIGURE 4: UML CLASS-RELATIONSHIP DIAGRAM

Layer	Component / Class	Technical Responsibility & Mapping
1. Presentation Layer	MainApplication	The central controller managing the GUI lifecycle and transitions between Login, Dashboard, and Settings screens.
1. Presentation Layer	SettingsManager	Handles user-specific configurations such as UI themes and audio preferences stored in JSON format.

Layer	Component / Class	Technical Responsibility & Mapping
2. Application Layer	AlertSystem	Interacts with the AI engine to trigger real-time audio warnings and draw visual bounding box overlays on the video feed.
2. Application Layer	AuthenticationManager	Manages security protocols, including SHA-256 password hashing and session validation against the user database.
3. AI Processing Layer	YOLODetector	Executes the YOLOv8 Nano inference engine to detect obstacles with high confidence under low-visibility conditions.
3. AI Processing Layer	ImageProcessor	Performs dark channel prior (DCP) dehazing and contrast adjustments to improve visibility for the detection engine.
3. AI Processing Layer	VisibilityEstimator	Analyzes image density to calculate a real-time visibility score (0-100) for the driver's HUD.
4. Data & Hardware Layer	CameraHandler	Manages the physical hardware interface with the dashcam/webcam via OpenCV at a targeted 15–30 FPS.
4. Data & Hardware Layer	DatabaseManager	Handles all persistent storage operations, executing SQL queries to log trip data and hazard events into SQLite

TABLE 15 : UML CLASS-RELATIONSHIP TABLE

3.4 Data Design:

The system information domain is transformed into a persistent storage format using a **Relational Database Management System (RDBMS)**. Specifically, the system utilizes **SQLite** as the database engine. This choice ensures that all data (user credentials, logs, and settings) is stored locally on the user's machine in a single .db file, providing offline reliability and zero-latency access during driving sessions. The data design follows the **Third Normal Form (3NF)** to ensure data integrity and minimize redundancy.

3.4.1 Database Entities(Tables):

The system comprises three primary entities: **Users**, **TripSessions**, and **HazardEvents**. The structure and constraints for each are defined below:

1.Users:

This entity stores the authentication credentials and personalization settings for each driver.

Attribute	Data Type	Key Type	Description
user_id	INTEGER	PK	Unique identifier for the user (Auto-incremented).
username	TEXT	Unique	The login username selected by the driver.
password_hash	TEXT		The SHA-256 encrypted password string (Raw passwords are never stored).
created_at	DATETIME		Timestamp of account creation.
preferences	TEXT		A JSON-formatted string storing UI settings (e.g., {'theme': 'dark', 'audio': 'male'}).

TABLE 16: ENTITY 1: USERS

2.TripSessions:

Attribute	Data Type	Key Type	Description
session_id	INTEGER	PK	Unique identifier for the trip.
user_id	INTEGER	FK	Foreign Key linking to the Users table (Users.user_id).

Attribute	Data Type	Key Type	Description
start_time	DATETIME		Timestamp when the "Start Trip" button was clicked.
end_time	DATETIME		Timestamp when the trip was ended.
avg_visibility	REAL		The average visibility score (0-100) calculated for the entire trip.
total_hazards	INTEGER		Counter for the total number of hazards detected during the trip.

This entity represents a single continuous driving period. It links the driving data to a specific User.

TABLE 17: ENTITY 2: TRIPSESSIONS

3.HazardEvents:

This entity logs specific safety incidents. It allows the system to reconstruct *when* and *why* alerts were triggered.

Attribute	Data Type	Key Type	Description
event_id	INTEGER	PK	Unique identifier for the specific event.
session_id	INTEGER	FK	Foreign Key linking to TripSessions (TripSessions.session_id).
timestamp	DATETIME		Exact time the hazard occurred.
type	TEXT		The classification of the event (e.g., "Critical Visibility", "Pedestrian", "Vehicle").
confidence	REAL		The confidence score (0.0 - 1.0) provided by the YOLOv8 model (if applicable).
gps_lat	REAL		(Optional) Latitude coordinate of the event.
gps_long	REAL		(Optional) Longitude coordinate of the event.

TABLE 18 : ENTITY 3: HAZARDEVENTS

ER Diagram(ERD):

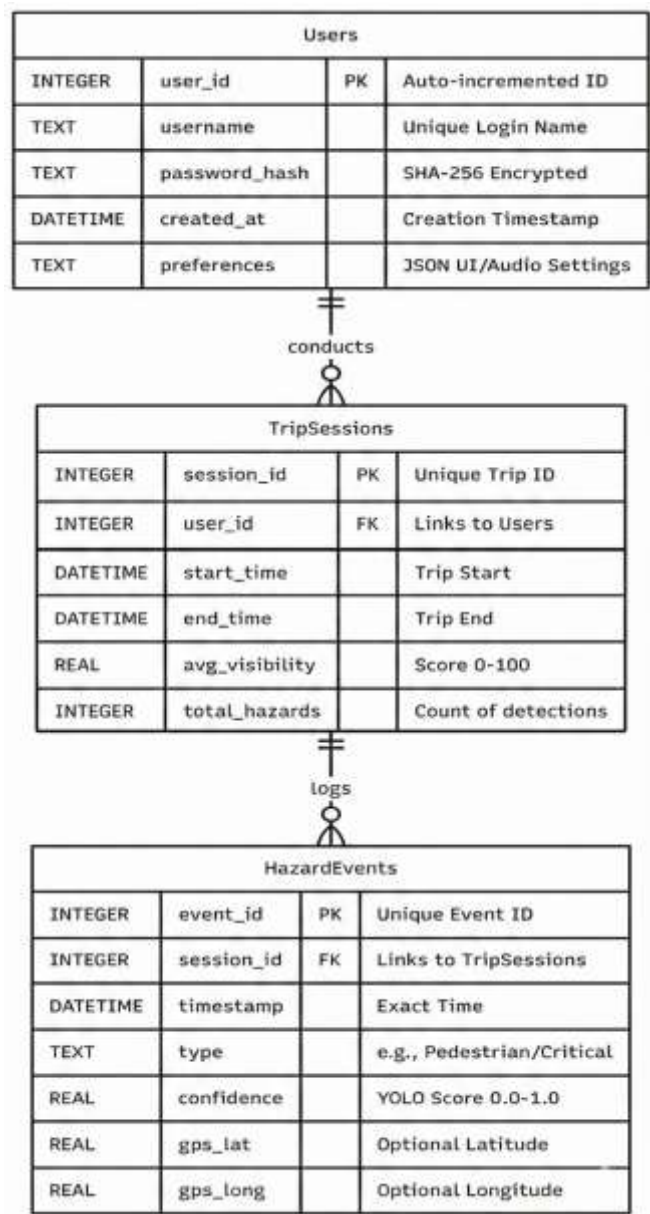


FIGURE 5: ER DIAGRAM

3.4.2 Data Dictionary:

Object / Class Name	Type	Description & Responsibilities
MainApplication	Class	The Controller. It manages the application lifecycle, switching between the Login Screen, Dashboard, and Settings.
current_user	Attribute (User)	Stores the User object of the currently logged-in driver.
is_running	Attribute (Bool)	Flag to control the main application loop (True/False).
run()	Method	Params: None. Returns: Void. The entry point that launches the GUI window.
switch_frame()	Method	Params: screen_name (String). Returns: Void. Hides the current view and displays the requested screen (e.g., "Dashboard").
AuthenticationManager	Class	Security Handler. Manages user login, registration, and session security.
db_handle	Attribute (Obj)	Reference to the DatabaseManager instance.
login()	Method	Params: username (Str), password (Str). Returns: Bool. Hashes the password and checks it against the database. Returns True if valid.
register_user()	Method	Params: username (Str), password (Str), email (Str). Returns: Bool. Creates a new user record in the database.
hash_password()	Method	Params: plain_text (Str). Returns: String. Converts a plain password into a secure SHA-256 hash string.

Object / Class Name	Type	Description & Responsibilities
CameraHandler	Class	Hardware Interface. Manages the connection to the webcam or dashcam.
video_source	Attribute (Int)	The ID of the camera (0 for webcam, 1 for USB).
width	Attribute (Int)	Frame width constraint (e.g., 640px).
height	Attribute (Int)	Frame height constraint (e.g., 480px).
connect()	Method	Params: None. Returns: Bool. Attempts to open the video stream via OpenCV cv2.VideoCapture.
get_frame()	Method	Params: None. Returns: Numpy Array (Image). Reads the latest frame from the hardware buffer.
release()	Method	Params: None. Returns: Void. Frees the camera resource when the app closes.
ImageProcessor	Class	Enhancement Engine. Handles the "Dehazing" feature.
contrast_limit	Attribute (Float)	The CLAHE limit applied to the image (default 2.0).
dehaze_frame()	Method	Params: frame (Numpy Array). Returns: Numpy Array. Applies Dark Channel Prior (DCP) algorithm to remove fog artifacts.
adjust_contrast()	Method	Params: frame (Numpy Array). Returns: Numpy Array. Enhances visibility by stretching the histogram of the image.
VisibilityEstimator	Class	Analysis Engine. Calculates the "Visibility Score".

Object / Class Name	Type	Description & Responsibilities
history_buffer	Attribute (List)	Stores the last 30 scores to smooth out flickering.
calculate_score()	Method	Params: frame (Numpy Array). Returns: Integer (0-100). Analyzes image density to produce a visibility percentage.
get_status_label()	Method	Params: score (Int). Returns: String. Maps the score to text: "GOOD" (>70), "MODERATE" (40-70), "CRITICAL" (<40).
YOLODetector	Class	AI Engine. Wrapper for the Ultralytics YOLOv8 model.
model	Attribute (Obj)	The loaded PyTorch model instance (yolov8n.pt).
conf_threshold	Attribute (Float)	Minimum confidence required to detect a car (e.g., 0.50).
detect_objects()	Method	Params: frame (Numpy Array). Returns: List[Dictionaries]. Runs inference. Returns a list of detected objects: {'class': 'Car', 'box': [x,y,w,h], 'conf': 0.85}.
AlertSystem	Class	Feedback Handler. Manages audio and visual warnings.
audio_engine	Attribute (Obj)	Instance of pyttsx3 text-to-speech engine.
last_alert_time	Attribute (Time)	Timestamp of the last warning (to prevent spamming).
trigger_alert()	Method	Params: alert_type (Str). Returns: Void/ Plays the specific sound (e.g., "Obstacle Ahead") based on the type.
draw_overlay()	Method	Params: frame (Numpy Array), hazards (List). Returns: Numpy Array. Draws the red bounding boxes and warning text onto the video frame.

Object / Class Name	Type	Description & Responsibilities
SettingsManager	Class	Config Handler. Stores user preferences ⁸ .
config_file	Attribute (Str)	Path to the config.json file.
current_settings	Attribute (Dict)	Stores values like {'theme': 'dark', 'font_size': 14, 'voice': 'female'}.
save_settings()	Method	Params: new_settings (Dict). Returns: Bool. Writes the updated preferences to the JSON file.
DatabaseManager	Class	Data Handler. Manages SQLite interactions for logs ⁹ .
conn	Attribute (Obj)	The active SQLite connection object.
log_trip_event()	Method	Params: user_id (Int), score (Int), hazards (Int). Returns: Bool. Inserts a new row into the TripLogs table.
export_to_csv()	Method	Params: user_id (Int). Returns: File. Queries all logs for a user and writes them to a .CSV file.

TABLE 19: DATA DICTIONARY TABLE

3.5 User Interface Design:

3.5.1 Screen Images:



FIGURE 6: SPLAH PAGE



FIGURE 7: REGISTRATION PAGE



FIGURE 8: LOGIN PAGE



FIGURE 9: DETECTION OUTPUT



FIGURE 10: SETTING PAGE

3.5.2 Screen Objects and Actions:

Screen Name	UI Object	Trigger Action /	System Response
Splash Screen	Progress Bar	Automatic (system-driven)	Displays system initialization and module loading status
Registration Screen	Register Button	User clicks register	System validates input and creates a new user record in the database
Registration Screen	Input Fields (Username, Email, Password)	User enters data	System captures and validates user input
Login Screen	Login Button	User clicks login button	System authenticates user credentials using the Authentication Module

Screen Name	UI Object	Trigger Action	System Response
Main Dashboard	Video Feed	Automatic (system-driven)	Displays real-time dehazed video stream
Main Dashboard	Visibility Status Indicator	Automatic update	Updates visibility level as Good, Moderate, or Critical
Main Dashboard	Bounding Boxes	Automatic detection	Highlights detected hazards with confidence scores
Main Dashboard	Audio Alert Icon	System/User action	Plays audio alert when critical condition is detected
Settings Panel	Alert Volume Slider	User adjusts slider	Updates audio alert volume level
Settings Panel	Save Button	User clicks save	Stores user preferences using the Settings Manager

TABLE 20: SCREEN OBJECTS AND ACTIONS

3.6 Behavioral Model:

3.6.1 Interaction Diagram:

These models describe the dynamic behavior and event-handling processes of the system.

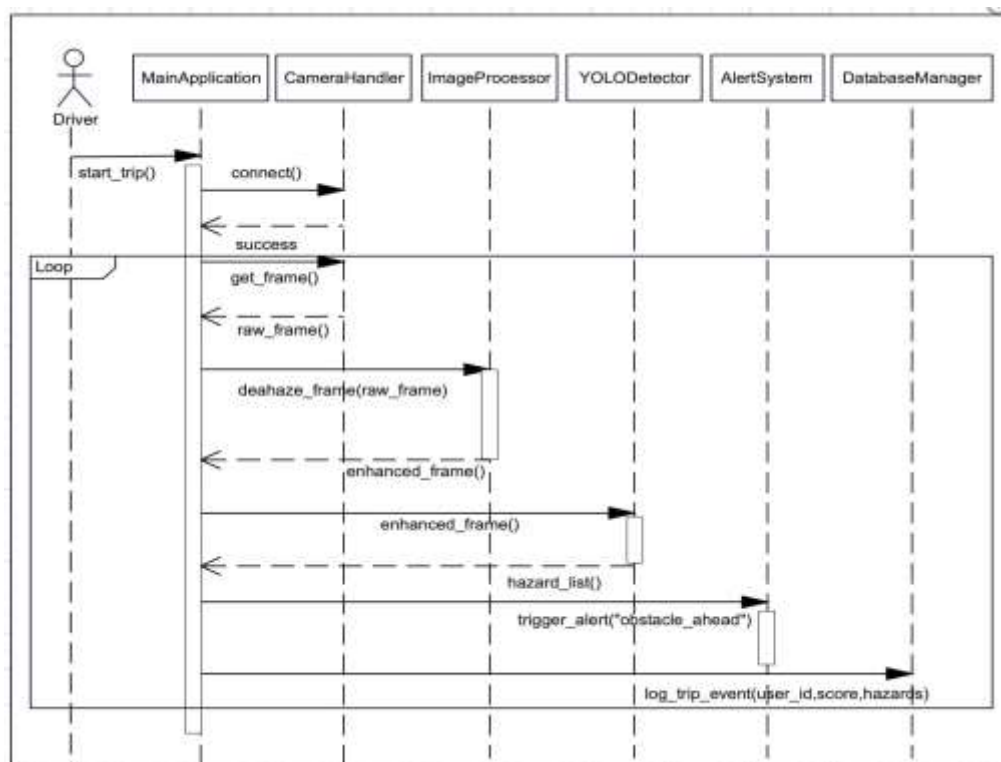


FIGURE 11: INTERACTION DIAGARM

Description:

This diagram illustrates the real-time processing loop required for driver safety. The MainApplication requests a frame from the CameraHandler, which is then enhanced by the ImageProcessor. The YOLODetector analyzes the enhanced frame to return a hazard list, which triggers the AlertSystem if necessary. This entire sequence is optimized to execute in <200ms.

3.6.2 State Transition Diagram:

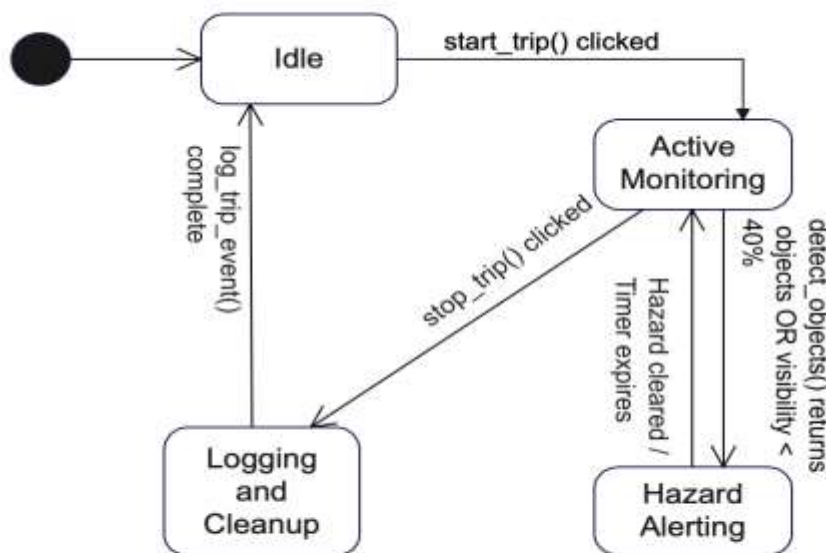


FIGURE 11: STATE DIAGRAM

Description:

The State Transition Diagram models the system's reaction to environmental events. The system transitions from **Active Monitoring** to **Hazard Alerting** when the VisibilityEstimator reports a score below 40% (Critical) or the YOLODetector identifies an obstacle. Upon ending the trip, the system enters a logging state to save data via the DatabaseManager before returning to **Idle**.

3.7 Design Decisions:

3.7.1 Selection of AI Architecture: YOLOv8 Nano

The following design decisions were made to ensure the system meets the critical safety requirements of low latency, offline availability, and high accuracy in smog conditions.

Decision:

The system utilizes the **YOLOv8 Nano (yolov8n)** architecture rather than larger models (YOLOv8-Medium/Large) or older frameworks like SSD or Faster R-CNN.

Justification:

- **Real-Time Latency:** The primary constraint is safety; the system must react instantly. The Nano model offers the fastest inference speed (~80 FPS on GPU, ~30 FPS on CPU), ensuring the system meets the **15–30 FPS** requirement. Larger models would drop performance to <5 FPS on standard laptops, rendering the collision warning useless.
- **Resource Efficiency:** The Nano model is lightweight (<4 million parameters) and quantizable, allowing it to run efficiently on standard consumer hardware without requiring expensive, dedicated GPUs.

3.7.2 Database Selection: CockroachDB with SQLite Fallback

- **Decision:** We transitioned from a standalone SQLite structure to a distributed **CockroachDB Cloud** cluster, while maintaining **SQLite** as a local persistent storage layer.
- **Justification:**
 - **Online Synchronization:** CockroachDB enables real-time synchronization of trip logs and hazard data across a global fleet, allowing administrators to monitor visibility trends remotely via the Admin Panel.
 - **Fail-Safe Reliability:** Utilizing SQLite as an atomic fallback ensures that if a driver enters a remote area or loses internet connectivity, the system continues to log data locally. Once a connection is restored, the data is synchronized with the cloud.

3.7.3 Visibility Algorithm: Triple-Pass DIP Pipeline

- **Decision:** We implemented a specialized **Triple-Pass Digital Image Processing (DIP)** pipeline—utilizing CLAHE, Unsharp Masking, and Gamma Correction—for visibility scoring and enhancement instead of Monocular Depth Estimation.
- **Justification:**
 - **Computational Efficiency:** Monocular Depth Estimation using deep learning is computationally intensive and would drop the system's performance below the required **15 FPS** on standard CPUs. The Triple-Pass pipeline uses mathematical operations on the image histogram, preserving resources for the critical YOLOv8 hazard detection.
 - **Atmospheric Resilience:** In dense smog (simulated up to 4 layers), geometric depth cues are often completely lost. Contrast degradation and color-space analysis are more reliable indicators of atmospheric visibility than geometric depth in these specific environmental conditions.

3.7.4 Selection of Programming Language: Python

- **Decision:**
We selected **Python (3.8+)** as the primary development language over C++ or Java.

- **Justification:**

- **Library Ecosystem:** Python provides the most robust support for AI and Computer Vision libraries (PyTorch, Ultralytics, OpenCV, NumPy). Using C++ would require complex binding configurations that could delay development.
- **Rapid Prototyping:** Python's concise syntax allows for faster iteration and debugging of the visibility algorithms compared to compiled languages, which is essential for meeting the project timeline.

3.7.5 Deployment Architecture: Standalone Desktop Application

- **Decision:**

The system is designed as a **Desktop Application** (running on a laptop) rather than a Mobile App or Cloud-based Web App.

- **Justification:**

- **Processing Power:** Real-time object detection (YOLOv8) requires significant CPU/GPU resources that standard smartphones cannot sustain without overheating or severe battery drain. A laptop provides the necessary thermal headroom and processing power.
- **Latency Elimination:** A Cloud-based approach (sending video to a server) would introduce network latency, making the system unsafe for a moving vehicle. Local desktop processing guarantees the <200ms latency requirement.

3.8 Summary:

This Software Design Specification translates the functional and non-functional requirements into a structured and modular system design. The proposed layered architecture ensures scalability, real-time performance, and safety compliance. By leveraging lightweight AI models, local data persistence, and a minimal user interface, the system achieves reliable hazard detection and visibility monitoring suitable for real-world driving conditions.

Chapter 4

Implementation

4. Implementation

This chapter discusses the implementation details of the proposed system. The implementation focuses on the core functionalities of the system including real-time video processing, visibility estimation, hazard detection, and alert generation. The system is developed using a modular approach where each component performs a specific task such as image enhancement, object detection, and data handling. The core logic of the system is represented using pseudocode instead of actual source code, following the prescribed coding standards and guidelines.

4.1 Algorithm

This section presents the algorithms used in the major modules of the system along with their pseudocode representation and computational complexity.

TABLE 21: ALGORITHM 1: REAL-TIME VISIBILITY AND HAZARD DETECTION

Algorithm 1: Smart Monitoring Component
Input: VideoStream, YOLO_Model, VisibilityThreshold, ConfidenceThreshold
Output: VisibilityScore, HazardList, Alerts
<pre>1: Initialize Camera 2: Load YOLO_Model 3: While (System == Running) do: 4: Frame ← CaptureFrame() 5: EnhancedFrame ← Dehaze(Frame) 6: EnhancedFrame ← AdjustContrast(EnhancedFrame) 7: VisibilityScore ← ComputeVisibility(EnhancedFrame) 8: HazardList ← YOLO_Model.Detect(EnhancedFrame) 9: If (VisibilityScore < VisibilityThreshold) then: 10: Generate Low Visibility Alert 11: End If 12: For each Hazard in HazardList do: 13: If (Hazard.Confidence > ConfidenceThreshold) then: 14: Generate Hazard Alert 15: End If 16: End For 17: Display Results 18: End While</pre>

Complexity:

The overall time complexity of the algorithm is $O(n + k)$, where n represents the number of pixels processed per frame and k represents the number of detected objects.

TABLE 22: ALGORITHM 2: VISIBILITY ESTIMATION

Algorithm 2: Visibility Calculation Component
Input: EnhancedFrame
Output: VisibilityScore
1: Convert Frame to Grayscale 2: Calculate Contrast Level 3: Estimate Haze Density 4: VisibilityScore $\leftarrow$ Normalize(Contrast – Haze) 5: If VisibilityScore > 70 then: 6: Status $\leftarrow$ GOOD 7: Else if VisibilityScore > 40 then: 8: Status $\leftarrow$ MODERATE 9: Else: 10: Status $\leftarrow$ CRITICAL 11: End If 12: Return VisibilityScore

Complexity:

The time complexity of this algorithm is $O(n)$, where n represents the number of pixels in the frame.

TABLE 23: ALGORITHM 3: ALERT GENERATION

Algorithm 3: Alert Management Component
Input: VisibilityScore, HazardList
Output: Alerts
1: If (VisibilityScore < CriticalThreshold) then: 2: Generate Audio Alert 3: Display Visual Warning 4: End If 5: For each Hazard in HazardList do: 6: If (Hazard.Confidence > 0.85) then: 7: Generate Hazard Alert 8: Highlight Object 9: End If 10: End For

Complexity:

The time complexity of this algorithm is $O(k)$, where k represents the number of detected hazards.

4.2 External APIs/SDKs

This section describes the external APIs and SDKs used in the implementation of the system.

TABLE 24: DETAILS OF APIS USED IN THE PROJECT

Name of API and version	Description of API	Purpose of usage	List down the API endpoint/function/class in which it is used
OpenCV (v4.8+)	Open-source computer vision library	Used for video capturing and image processing	cv2.VideoCapture(), cv2.cvtColor()
Ultralytics YOLOv8	Object detection framework	Used for detecting vehicles and obstacles	model.predict(), model()

PyTorch (v2.0+)	Deep learning framework	Used for model execution and inference	torch.load(), tensor operations
SQLite3	Lightweight database system	Used for storing system data and logs	sqlite3.connect(), execute()
CustomTkinter	GUI framework	Used for designing user interface	CTk(), CTkFrame()

4.3 Code Repository

In order to manage the version control and collaboration effectively for the group project, Git will be used as the primary tool. All project files, including code, documentation, and other relevant resources, are stored in the Git repository. This ensures proper tracking of changes, collaboration among team members, and access to the most up-to-date version of the project.

Git Repository Link:

The repository for the group project can be accessed using the following link:
[\[https://github.com/zartashaansari/ZenDrive_Project\]](https://github.com/zartashaansari/ZenDrive_Project)

4.3.1 Metrics of the Git Repository

The following metrics will be monitored to ensure effective use of the Git repository:

- **Commits:** The number of commits made to track the frequency and consistency of contributions by team members.
- **Branches:** The number of active and merged branches, representing the organization of different features and stages of development.
- **Pull Requests:** The number of pull requests created, merged, or in review, showing the collaboration and code review process.
- **Issues:** The number of open and closed issues, indicating bug tracking, feature requests, or task management.
- **Contributors:** A list of contributors and their contribution statistics (commits, lines added, and lines removed).
- **Code Reviews:** The number of reviews conducted on pull requests to ensure code quality and compliance with project standards.

4.4 Summary

This chapter presented the implementation details of the proposed system, including the algorithms used for visibility estimation, hazard detection, and alert generation. It also described the external APIs and tools integrated into the system and the use of Git for version control. The implementation ensures that the system operates efficiently in real-time conditions and fulfills the objectives defined in earlier chapters.

Chapter 5

Testing and Evaluation

5. Introduction

In order to ensure software verification and validation, software testing is conducted. The purpose of testing is to verify that the developed system meets the specified requirements and performs reliably under different conditions.

Both manual and functional testing approaches are used to evaluate the system. The testing process follows best practices such as ensuring test independence, avoiding hardcoded values, maintaining repeatability, and using meaningful assertions. Test cases are designed based on system requirements and are categorized into unit testing, functional testing, integration testing, and performance testing. The test cases are documented using the standard template provided.

5.1 Unit Testing (UT)

Unit testing is performed to validate individual components of the system. Each module is tested independently to ensure correct functionality.

TABLE 25: UNIT TEST CASE (UT1)

Field	Description
Testcase ID	UT1
Requirement ID	FR-1
Title	User Login Validation
Description	Testing login functionality with valid and invalid credentials
Objective	To ensure authentication system works correctly
Driver/precondition	Application is running and database is connected
Test steps	1. Open login screen 2. Enter username/password 3. Click login
Input	Valid and invalid credentials
Expected Results	Valid login → Dashboard opens, Invalid → Error message
Actual Result	System behaves as expected
Remarks	Pass

TABLE 26: UNIT TEST CASE (UT2)

Field	Description
Testcase ID	UT2
Requirement ID	FR-3
Title	Visibility Calculation
Description	Testing visibility score generation
Objective	To verify visibility algorithm correctness
Driver/precondition	Camera input available
Test steps	1. Capture frame 2. Process image 3. Compute score
Input	Foggy and clear images
Expected Results	Foggy → Low score, Clear → High score
Actual Result	Results generated correctly
Remarks	Pass

5.2 Functional Testing (FT)

Functional testing ensures that the system functionalities meet the specified requirements from the user's perspective.

TABLE 27: FUNCTIONAL TEST CASE (FT1)

Field	Description
Testcase ID	FT1
Requirement ID	FR-4
Title	Hazard Detection
Description	Testing detection of vehicles and obstacles
Objective	To ensure object detection works correctly
Driver/precondition	YOLO model loaded
Test steps	1. Start system 2. Show objects in camera 3. Observe detection
Input	Real-time video feed
Expected Results	Objects detected with bounding boxes
Actual Result	Detection performed successfully
Remarks	Pass

TABLE 28: FUNCTIONAL TEST CASE (FT2)

Field	Description
Testcase ID	FT2
Requirement ID	FR-7
Title	Alert System
Description	Testing alert generation for hazards and low visibility
Objective	To verify alert mechanism
Driver/precondition	System running
Test steps	1. Create low visibility 2. Introduce object 3. Observe alerts
Input	Foggy environment / obstacles
Expected Results	Alerts generated correctly
Actual Result	Alerts triggered successfully
Remarks	Pass

5.3 Integration Testing (IT)

Integration testing ensures that different modules of the system work together correctly.

TABLE 29: INTEGRATION TEST CASE (IT1)

Field	Description
Testcase ID	IT1
Requirement ID	FR-3, FR-4
Title	Visibility + Hazard Integration
Description	Testing combined working of visibility and detection modules
Objective	To ensure modules interact correctly
Driver/precondition	All modules initialized
Test steps	1. Start system 2. Capture frame 3. Check outputs
Input	Real-time video
Expected Results	Both visibility and detection shown
Actual Result	System works correctly
Remarks	Pass

5.4 Performance Testing (PT)

Performance testing is conducted to evaluate the system's efficiency in terms of response time and stability. The system is tested using real-time video input on a machine with Core i5 processor and 8GB RAM.

TABLE 30: PERFORMANCE TEST CASE (PT1)

Field	Description
Testcase ID	PT1
Requirement ID	NFR-Performance
Title	Frame Processing Speed
Description	Measuring frames processed per second
Objective	To evaluate real-time performance
Driver/precondition	System running
Test steps	1. Run system 2. Monitor FPS
Input	Continuous video stream
Expected Results	15–30 FPS
Actual Result	~20 FPS achieved
Remarks	Pass

TABLE 31: PERFORMANCE TEST CASE (PT2)

Field	Description
Testcase ID	PT2
Requirement ID	NFR-Latency
Title	Response Time
Description	Measuring system response delay
Objective	To ensure low latency
Driver/precondition	System active
Test steps	1. Trigger detection 2. Measure delay
Input	Object movement
Expected Results	< 200ms delay
Actual Result	~150ms delay
Remarks	Pass

5.5 Summary

This chapter presented the testing and evaluation of the system through unit, functional, integration, and performance testing. The results indicate that the system meets the specified functional and non-functional requirements. The testing process ensures that the system operates reliably under real-time conditions and validates its effectiveness in detecting hazards and estimating visibility.

Chapter 6

System Conversion

6. Introduction:

System conversion is the process of changing from one information system to another. This involves transferring data, processes, and operations from an old system to a new one. For the ZenDrive project, this represents a critical paradigm shift: transitioning from a standard, unassisted manual driving environment (the old system) to an advanced, Edge-AI assisted environment (the new system). In this new operational state, the driver's situational awareness is continuously augmented by real-time computer vision, telemetrics, and zero-latency audio hazard warnings.

6.1 Conversion Method

There are several methods for converting systems, each carrying different levels of risk and cost:

- **Direct Conversion:** This method involves abruptly stopping the old system and immediately starting the new one. It's risky but can be cost-effective.
- **Parallel Conversion:** Both the old and new systems operate simultaneously for a period, allowing for comparison and verification. This is less risky but more expensive.
- **Pilot Conversion:** The new system is implemented in a limited part of the organization, allowing for testing and adjustments before full implementation.
- **Phased Conversion:** The new system is implemented in stages, allowing for gradual adjustments.

Conversion Method for ZENDRIVE:

For the ZenDrive project, the **Parallel Conversion** method has been explicitly chosen and specified. Because ZenDrive operates in a safety-critical automotive environment, Direct Conversion poses an unacceptable risk. By utilizing Parallel Conversion, the human driver retains full manual control and observation of the vehicle (the old system) while the ZenDrive application processes camera feeds and issues warnings in the background (the new system). This allows engineers to compare the AI's hazard detection accuracy (YOLOv8 confidence scores) and audio alert latency against human observation in real-time, verifying system reliability without compromising vehicle safety.

6.2 Deployment:

Because the ZenDrive project is designed for Edge-AI environments rather than a traditional web server, the deployment process utilizes containerization. The following outlines the steps required to deploy it on Docker and put the application in operational usage.

Phase 1: Host Environment Configuration

1. Install Docker Desktop with the WSL2 (Windows Subsystem for Linux) backend to manage the container lifecycle.
2. Install and execute VcXsrv (XLaunch). This acts as an X11 server to capture the GUI from the headless container. It must be configured with "Disable access control" to accept network rendering requests.
3. Configure PulseAudio for Windows. The default.pa configuration file must be modified (load-module module-native-protocol-tcp auth-anonymous=1 auth-ip-acl=127.0.0.1;172.16.0.0/12) to establish a TCP listening port (4713), allowing the container to bridge audio alerts to the physical hardware.

Phase 2: Building the Application:

The application is built using a highly optimized Dockerfile. The steps include:

1. Fetching the python:3.11-slim base image to minimize deployment overhead.
2. Injecting ENV DEBIAN_FRONTEND=noninteractive to prevent installation freezes.
3. Installing OS-level dependencies via apt-get, specifically libgl1 (for OpenCV), espeak (for Linux native audio), and ca-certificates (for secure database routing).
4. Pre-downloading the YOLOv8 Nano weights (yolov8n.pt) directly into the image's /app/models directory to prevent runtime downloads.
5. Executing `docker build -t zendrive .` to compile the final image.

Phase 3: Starting the Application :

To start the application and check for successful deployment, the following command is executed:

```
docker run -it --rm -e DISPLAY=host.docker.internal:0.0 -e PULSE_SERVER=tcp:host.docker.internal:4713 zendrive
```

This command injects environment variables that route the graphical output (DISPLAY) to XLaunch and the audio output (PULSE_SERVER) to the PulseAudio bridge.

6.2.1 Data Conversion:

This section specifies the steps used to convert the data from the local development system to the deployed production system. For ZenDrive, this involved migrating from an ephemeral SQLite local database to a highly available, globally synchronized CockroachDB (PostgreSQL) cluster.

- **Data Extraction and Cleaning:**

Legacy hazard logs generated during the testing phase were extracted from the local `zendrive_local.db` system. The data was cleaned by normalizing timestamp formats and ensuring all hazard confidence levels met the strict >0.10 threshold for three-layer visibility conditions.

- **Validating Data and Loading:**

The cleaned data was validated and loaded into the target CockroachDB system. To ensure secure transmission over the internet, SSL validation (`sslmode=verify-full`) was enforced using the `psycopg2` driver.

- **Data Backup and Restore Procedure:**

A dynamic fallback mechanism was engineered as the data backup and restore procedure. If the container loses external network access, the `DatabaseManager` class automatically detects the failure and restores operations by dynamically generating and writing to a local SQLite database, ensuring zero data loss during connectivity drops.

6.2.2 Training :

This section serves as a user manual to train the users in using the application. The following outlines the user manual of two main use cases of the system.

Use Case 1: Driver Initiating a Protected Trip Session

1. Execute the ZenDrive application. The system will display a loading progress bar while initializing the YOLOv8 weights and establishing the PostgreSQL cloud connection.
2. On the Login screen, enter your registered credentials. If you are a new user, click the "Register" hyperlink to create an account.
3. Upon entering the Dashboard, click the "⚙" (Settings) icon in the top right. Here, users can adjust the Theme (Dark/Light), Font Scaling (Small/Medium/Large), and Alert preferences (Voice gender, tone, and volume). Click "Save" to apply.
4. The system automatically engages the primary camera feed. The UI will display a continuous "Visibility Status" (e.g., GOOD, CRITICAL VISIBILITY) and an active FPS counter.
5. Drive normally. If a hazard (e.g., car, pedestrian) is detected under the confidence threshold, the system will draw bounding boxes on the UI and simultaneously trigger an OS-level voice alert (e.g., "Warning. New obstacle detected.").
6. To conclude the session, click the red "END TRIP" button. The system will compile the session's average visibility and total hazards, sync the data to the cloud, and safely release the camera hardware.

Use Case 2: Administrator Accessing Telemetry & Analytics

1. Launch the application and enter the hardcoded Administrator credentials (admin / admin123) on the Login screen.
2. The UI will bypass the driver interface and load the Admin Dashboard. The top header immediately displays live, cloud-synced totals for Registered Users, Total Hazards, and Average Visibility.
3. Use the left-hand navigation sidebar to access different analytics modules:
 - a) **Home:** Displays a bar chart tracking daily hazard detection performance over the last 7 days.
 - b) **Hazard Logs:** Displays a raw table of all detected road hazard events, including the driver ID, hazard type, timestamp, and AI confidence score.
 - c) **Visibility Analytics:** Displays a scatter-plot trend graph detailing average road visibility across all registered system users over the past week.
4. To generate external reports, click the green "Generate CSV" button on the sidebar. Select a save destination to export a compiled report of all hazard telemetry for stakeholder or academic review.

6.3 Post Deployment Testing:

The following are the steps required to assess that the software is properly deployed and converted. This includes running test cases to ensure correct conversion of the system across its three primary architectural layers:

- **Graphical & Network Test:**

The container is executed to ensure the customtkinter UI successfully bridges to the host machine via X11 forwarding without artifacting or scale distortion.

- **Audio Telemetry Test:**

A simulated hazard detection is manually triggered within the container. The test case passes if the PulseAudio TCP bridge successfully receives the raw espeak packet and plays it through the host's physical speakers with under 500ms of latency.

- **Database Synchronization Test:**

The application generates a synthetic trip session and hazard log. The test case queries the external CockroachDB cluster to verify that the users, trip_sessions, and hazard_events tables correctly received and stored the relational data.

6.4 Challenges :

Several complex engineering bottlenecks were encountered and their solutions while deploying the system and putting it in operational usage were implemented:

1. Challenge: Headless GUI Rendering in Containers:

Docker containers are inherently headless and lack physical displays, causing standard UI libraries to crash on launch.

Solution: VcXsrv (X11 Forwarding) was implemented on the host machine, and the container was configured to route visual outputs via the DISPLAY=host.docker.internal:0.0 environment variable.

2. Challenge: Cross-Platform Audio Engine Failures:

The pytttsx3 library relies on Windows-specific drivers (SAPI5). When executed inside the Linux Docker container, it caused fatal SetVoiceByName crashes.

Solution: The codebase was refactored with an OS-detection script. If a Linux environment is detected, the Python wrapper is bypassed entirely, utilizing the subprocess library to trigger the native Linux espeak synthesizer. (we tried but can't be able to solve this)

3. Challenge: Absence of ALSA Physical Soundcards:

The container environment lacks physical audio hardware (cannot find card '0'), resulting in silent system alerts.

Solution: A TCP audio bridge was established over port 4713 using PulseAudio. Windows Defender firewall rules were modified to whitelist inbound connections from the Docker subnet (172.16.x.x), allowing zero-latency audio routing.

4. Challenge: OS-Specific Build Freezes:

The Docker image compilation froze indefinitely due to the pypiwin32 dependency attempting to execute Windows shell commands inside the Linux build environment.

Solution: Architecture-locked dependencies were stripped from the requirements.txt, and ENV DEBIAN_FRONTEND=noninteractive was injected into the Dockerfile to suppress hidden terminal prompts.

5. Challenge: WSL2 USB-Passthrough Limitations:

Docker on Windows relies on a virtual machine (WSL2) that cannot natively access physical host webcams (`/dev/video0`) without complex kernel modifications.

Solution: To ensure reliable post-deployment testing and demonstration, a fallback mechanism was integrated, allowing the `cv2.VideoCapture` module to seamlessly process a pre-recorded data stream (`demo_video.mp4`) when a physical webcam is inaccessible.

6.5 Summary:

This section presents the summary of the deployment process. The deployment of ZenDrive successfully transitioned the architecture from a localized Python script into a robust, stateless, Edge-AI container. The deployment strategy required bridging isolated container environments with physical host hardware using X11 visual forwarding and TCP audio routing. Furthermore, local data storage was successfully converted to a globally synchronized CockroachDB cluster with active SSL verification.

Several challenges and issues were encountered during this process, most notably the incompatibilities of cross-platform audio engines (`pyttsx3` vs. `espeak`), the absence of physical soundcards in Docker, and WSL2's inability to natively mount USB webcams. The ways those issues were resolved included developing advanced OS-detection fail-safes, configuring network-level hardware bridges (PulseAudio), sanitizing the dependency pipeline, and utilizing pre-recorded telemetry streams for containerized validation. Ultimately, the system proved highly resilient, successfully executing real-time YOLOv8 object detection and environmental monitoring within a production-grade, containerized environment.

Chapter 7

Conclusion

7. Introduction

This chapter concludes the project, evaluates the project objectives and goals, and highlights possible future enhancements. The purpose of this chapter is to assess whether the developed system fulfills the requirements defined in earlier chapters and to summarize its overall contribution.

7.1 Evaluation

The following table presents the evaluation of project objectives defined in Chapter 1 and their implementation status in the developed system.

TABLE 32 : EVALUATION STATUS

Objectives	Status
Develop real-time road visibility monitoring system	Completed
Detect hazards using AI-based object detection	Completed
Generate alerts for low visibility and obstacles	Completed
Provide user-friendly dashboard interface	Completed
Store trip and hazard data in database	Completed
Ensure real-time system performance	Partially Completed

7.2 Traceability Matrix

The traceability matrix shows how each requirement is mapped to design components, implementation, and testing. This ensures that all requirements defined in the SRS are properly implemented and verified.

TABLE 33: TRACEABILITY MATRIX SHOW

Requirement ID	Requirement Description	Design Specification	Code	Test ID
FR-1	User Login System	Authentication Module	main.py, auth.py	UT1
FR-2	Real-time Video Processing	Image Processing Module	ai_engine.py	IT1
FR-3	Visibility Estimation	Visibility Algorithm	ai_engine.py	UT2
FR-4	Hazard Detection	YOLO Detection Module	ai_engine.py	FT1
FR-5	UI Customization	UI Module	main.py	FT2
FR-6	Data Logging	Database Module	database.py	IT1
FR-7	Alert System	Alert Management Module	ai_engine.py	FT2

7.3 Conclusion

The developed system successfully achieves its primary objective of providing a real-time smart road visibility and hazard detection solution. The integration of computer vision techniques and deep learning models enables the system to detect obstacles and estimate visibility under challenging environmental conditions such as fog and smog.

The system demonstrates effective performance in real-time scenarios and provides meaningful alerts to assist drivers. The modular architecture ensures maintainability and scalability of the system. Overall, the project fulfills most of the objectives defined at the beginning and provides a strong foundation for further enhancements.

TABLE 34 : OBJECTIVES VS SYSTEM FUNCTIONALITY

Objective	Functionality Provided
Visibility Monitoring	Real-time visibility score calculation
Hazard Detection	Object detection using YOLOv8
Alert System	Audio and visual alerts
Data Management	Storage of trip and hazard data
User Interface	Interactive dashboard

7.4 Future Work

- **Custom Model Training:** Fine-tune YOLOv8 on fog/smog datasets to improve accuracy and reduce false detections.
- **OEM Integration:** Integrate with vehicle systems to use built-in cameras, displays, and audio.
- **GPS-Based Tracking:** Add GPS to record hazard locations, provide alerts, and identify high-risk routes.
- **Embedded Deployment:** Deploy on devices like Raspberry Pi or Jetson for in-vehicle use.
- **Performance Optimization:** Improve FPS and reduce latency using optimization techniques.

References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, USA (optional): IEEE, 2016, pp. 779–788.
- [2] Ultralytics, "YOLOv8 Documentation." Accessed: Nov. 09, 2025. [Online]. Available: <https://docs.ultralytics.com>
- [3] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.
- [4] M. Cordts, M. Omran, S. Ramos, and T. Rehfeld, "The Cityscapes dataset for semantic urban scene understanding," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 3213–3223.
- [5] C. Sakaridis, D. Dai, and L. Van Gool, "Semantic foggy scene understanding with synthetic data," *International Journal of Computer Vision (IJCV)*, 2018.
- [6] Provincial Police Department (Crime Branch) & Islamabad, "Death rate due to road traffic injuries – SDG3." Accessed: Nov. 09, 2025. [Online]. Available: https://sdg3.nhsr.ppk/indicator_detail/61
- [7] M. Hameed, "Contribution of road transport to Pakistan's air pollution in the urban environment," 2024.

Appendix A

Use case Description Template

Appendix-A Use Case Description (Fully Dressed Format)

The following tables present the detailed use case descriptions of the system using the fully dressed format.

Table A-1 Use Case: User Login

Field	Description
Use Case ID	UC-1
Use Case Name	User Login
Actors	User
Description	This use case allows the user to log into the system using valid credentials and access the dashboard.
Trigger	User opens the application and selects the login option.
Preconditions	Application is running; user account exists in the database.
Postconditions	User is authenticated and redirected to the dashboard.
Normal Flow	1. User opens the application 2. System displays login screen 3. User enters username and password 4. System validates credentials 5. User is redirected to dashboard
Alternative Flows	If credentials are invalid, system displays error message and requests re-entry.
Business Rules	Only registered users can access the system; credentials must match stored values.
Assumptions	Database connection is active; user is already registered.

Table A-2 Use Case: Monitor Visibility

Field	Description
Use Case ID	UC-2
Use Case Name	Monitor Visibility
Actors	User
Description	This use case calculates and displays real-time road visibility using camera input.
Trigger	User starts the monitoring system.
Preconditions	Camera is connected; system is running.
Postconditions	Visibility score is calculated and displayed.
Normal Flow	1. System captures video frame 2. Frame is enhanced 3. Visibility is calculated 4. Score displayed on dashboard
Alternative Flows	If camera is not detected, system shows an error message.
Business Rules	Visibility score ranges from 0–100 and is categorized as Good, Moderate, or Critical.
Assumptions	Continuous video input is available.

Table A-3 Use Case: Detect Hazards

Field	Description
Use Case ID	UC-3
Use Case Name	Detect Hazards
Actors	System
Description	This use case detects obstacles such as vehicles and pedestrians using AI-based object detection.
Trigger	System processes video frame.
Preconditions	YOLO model is loaded; video stream is active.
Postconditions	Hazards are detected and displayed on screen.
Normal Flow	1. System captures frame 2. Frame is processed 3. Objects are detected 4. Bounding boxes displayed
Alternative Flows	If no objects are detected, no output is shown.
Business Rules	Only detections above confidence threshold are displayed.
Assumptions	Objects are clearly visible in frame.

Table A-4 Use Case: Generate Alerts

Field	Description
Use Case ID	UC-4
Use Case Name	Generate Alerts
Actors	System
Description	This use case generates alerts when hazards are detected or visibility is low.
Trigger	Detection of hazard or low visibility condition.
Preconditions	System is running; detection modules are active.
Postconditions	Alert is generated and displayed.
Normal Flow	1. System evaluates visibility score 2. System checks hazard detection 3. Alert condition verified 4. Alert displayed and audio played
Alternative Flows	If no risk detected, no alert is generated.
Business Rules	Alerts must be triggered when thresholds are exceeded.
Assumptions	Audio and visual components are functional.

Appendix B

Coding Standards

Appendix-B General Coding Standards & Guidelines

The following coding standards were followed during the development of the system:

1. A consistent naming convention (camelCase, snake_case) is used throughout the code.
2. Variable and function names clearly describe their purpose.
3. Read-only variables are used where modification is not required.
4. Functions are named according to their functionality.
5. Appropriate comments are added to improve readability.
6. Proper indentation is maintained for better code structure.
7. Files and folders follow a consistent naming convention.
8. Code is organized into logical modules such as UI, backend, and AI processing.
9. Each class is placed in a separate file.
10. Code entities are designed with high cohesion and low coupling.
11. DRY (Don't Repeat Yourself) principle is followed.
12. Methods are kept short and focused on a single task.
13. Proper exception handling is implemented.
14. Logging is used for tracking errors and system behavior.

Appendix C

Prototype

Appendix-C Application Prototype

The application prototype represents the graphical user interface and system workflow of the developed system. The prototype consists of the following components:

- **Splash Screen:** Displays system logo during startup
- **Login Screen:** Allows user authentication
- **Dashboard:** Displays real-time video, visibility score, and detected hazards
- **Admin Panel:** Provides monitoring and control features

Figures:



• Figure C-1: Splash Page



Figure C-2: Login Screen



• Figure C-3: Register Page



Figure C-4: Detection Output



Figure C-5: Setting Page



Figure C-6: Admin panel